Fundamentos de PyBullet

Análisis Arquitectónico del Entorno de Simulación

Clase de Robótica

March 19, 2026

Antes de Comenzar

La GTC 2026 de NVIDIA, la conferencia anual más importante en inteligencia artificial y computación avanzada, sorprendió a sus asistentes este año con la aparición de Olaf, el personaje de la película Frozen, desplazándose de manera autónoma sobre el escenario.

Figure: CEO de NVIDIA, Jensen Huang junto a Olaf

Link Video Conferencia

1. Importación de Librerías Base

```python
import pybullet as p
import pybullet_data
import time
```

Análisis:

- `pybullet`: El motor físico principal. Nos da acceso a la API (conexión, carga de modelos, cinemática, dinámica).
- `pybullet_data`: Una librería auxiliar esencial. PyBullet no sabe dónde están los archivos 3D por defecto; esta librería contiene las rutas a modelos estándar (como planos, mesas, o robots como el R2D2).
- `time`: Librería estándar de Python. La usaremos para sincronizar el tiempo de la CPU con el tiempo del simulador.

2. Arquitectura Cliente-Servidor: Conexión

```
# p.GUI para interfaz grafica, p.DIRECT para background
physicsClient = p.connect(p.GUI)
```

Análisis:

- PyBullet funciona bajo una arquitectura Cliente-Servidor. El motor de físicas (Servidor) corre de fondo, y nuestro script de Python (Cliente) le envía comandos.
- `p.GUI`: Inicia el servidor con renderizado visual y herramientas de depuración (vital para aprender y visualizar).
- `p.DIRECT`: Inicia sin interfaz gráfica. Es muchísimo más rápido. Se usa en etapas avanzadas (ej. entrenar Inteligencia Artificial) donde solo importan los cálculos matemáticos, no los gráficos.
- `physicsClient`: Guarda el ID (un número entero) de nuestra conexión por si manejamos múltiples simulaciones a la vez.

3. Resolución de Rutas (Paths)

```
# Ruta a modelos 3D por defecto
p.setAdditionalSearchPath(pybullet_data.getDataPath())
```

Análisis:

- Por seguridad y modularidad, el motor físico no busca archivos URDF (Universal Robot Description Format) en todo tu disco duro.
- Con esta línea, agregamos el directorio de `pybullet_data` a las rutas permitidas.
- Sin esto, cuando intentemos cargar un modelo básico como el suelo, el programa arrojará un error de "Archivo no encontrado".

4. Leyes de la Física: La Gravedad

```
# Configurar la gravedad (eje X, eje Y, eje Z)
p.setGravity(0, 0, -9.81)
```

Análisis:

- La gravedad no existe por defecto en un motor físico vacío; debemos declararla como un vector tridimensional (G_x, G_y, G_z).
- En PyBullet, la convención espacial dicta que el eje Z es "hacia arriba" (a diferencia de otros motores donde es el eje Y).
- Aplicamos $-9.81\, m/s^2$ en el eje Z para emular la gravedad terrestre.
- *Nota:* Si queremos simular un robot en el espacio o en la luna, simplemente alteramos este vector.

5. Instanciación del Entorno (El Suelo)

```
# Cargar un plano que servira como nuestro "suelo"
planeId = p.loadURDF("plane.urdf")
```

Análisis:

- `loadURDF` es la función más importante para poblar la simulación. Lee la geometría, masa y colisiones de un archivo XML.
- `"plane.urdf"` es un modelo predeterminado que carga un plano infinito para que los objetos no caigan al vacío.
- `planeId`: Cada vez que instanciamos un objeto, PyBullet le asigna un número identificador único (0, 1, 2...). Guardar este ID es fundamental si luego queremos alterar ese objeto específico.

6. El Motor del Tiempo: Bucle de Simulación

```
# Bucle infinito para mantener el programa vivo
while True:
    p.stepSimulation()
```

Análisis:

- Un simulador físico es como una película: es una sucesión rápida de fotos (frames).
- PyBullet **no avanza el tiempo de forma automática**. Eres tú, como programador, quien tiene el control del "reloj".
- `stepSimulation()`: Le dice al motor "Calcula todas las colisiones, fuerzas y movimientos para el siguiente instante de tiempo (t + Δt)".

7. Sincronización Real vs Simulación

```
# PyBullet calcula la fisica a 240 Hz
time.sleep(1./240.)
```

Análisis:

- Por defecto, cada `stepSimulation()` avanza $\Delta t = \frac{1}{240}$ segundos en el mundo virtual.
- Las computadoras modernas procesan esto casi instantáneamente. Si no pausamos el bucle, la simulación correría a miles de pasos por segundo y veríamos todo en "cámara súper rápida".
- `time.sleep(1/240)` le dice a nuestra CPU que espere esa fracción de segundo real, sincronizando el tiempo de nuestro mundo con el tiempo del simulador para que sea visible al ojo humano.

8. Liberación de Recursos

```
# Desconectar al terminar (buena practica)
p.disconnect()
```

Análisis:

- Aunque al cerrar la ventana o hacer *Ctrl+C* el sistema operativo mata el proceso, es una excelente práctica de ingeniería de software cerrar la conexión ordenadamente.
- Esto destruye el servidor de físicas, libera la memoria RAM utilizada por los modelos 3D y cierra los puertos de comunicación interna.

Ejecutando el Primer Plano

Desde nuestro repositorio, al ejecutar el codigo **cargando_un_plano.py** , observaremos :

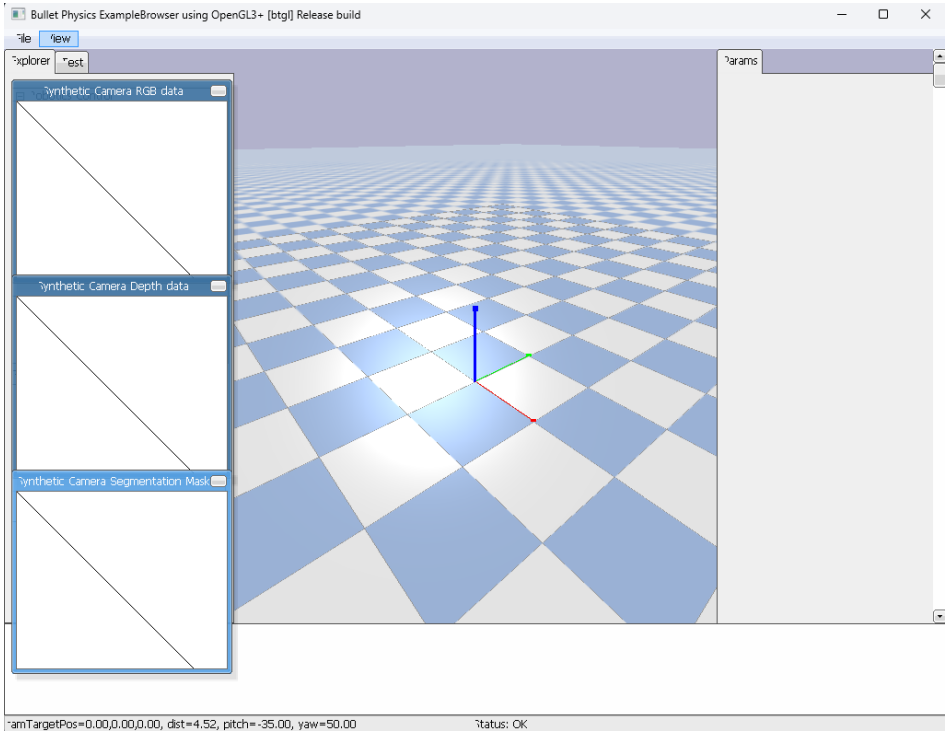


Figure: Primera vista de PyBullet

El Control de la Cámara (Manual)

¿Cómo me muevo en el espacio 3D con el ratón?

- **Zoom (Acercar/Alejar):** Rueda del ratón (Scroll).
- **Orbitar (Rotar la perspectiva):** `Ctrl` + `Clic Izquierdo` + Arrastrar el ratón. Esto hace que la cámara gire alrededor del centro de la pantalla.
- **Panorámica (Traslación):** `Ctrl` + `Clic Derecho` + Arrastrar el ratón. Esto mueve el centro de la cámara (el objetivo) hacia arriba o abajo por el plano.
- **Mover todo:** `Ctrl` + `Scroll`. Esto mueve toda la pantalla.

Control de la Cámara desde el Código

```
# Configurar la posicion inicial de la camara
distancia = 3.0
yaw = 45.0          # Rotacion horizontal (izq/der)
pitch = -30.0       # Rotacion vertical (arriba/abajo)
objetivo = [0, 0, 0] # Punto al que mira la camara (X, Y, Z)

p.resetDebugVisualizerCamera(distancia, yaw, pitch, objetivo
    )
```

Análisis Arquitectónico:

- No siempre queremos mover la cámara a mano cada vez que ejecutamos el código.
- `resetDebugVisualizerCamera`: Fuerza al renderizador (GUI) a posicionar la cámara virtual según coordenadas polares.
- **Yaw**: Gira la cámara alrededor del eje Z (como mirar a los lados).
- **Pitch**: Ángulo de inclinación. Un valor negativo (ej. -30) significa que miramos de arriba hacia abajo.

¿Puedo cargar otros planos?

```
# Cargar un estadio en lugar de un plano infinito
entornoId = p.loadURDF("samurai.urdf")
```

Análisis:

- ¡Sí! El archivo `"plane.urdf"` es solo un modelo 3D muy simple con forma de caja plana. No tiene nada de mágico.
- PyBullet incluye otros entornos dentro de `pybullet_data`.
- El motor físico trata al plano como cualquier otro objeto estático (masa $= 0$). Las colisiones de tu robot funcionarán igual contra el asfalto de un estadio que contra el plano básico.

¿Qué planos existen en PyBullet?

El Concepto Clave:

- PyBullet es un motor físico, no una base de datos de modelos.
- No hay un comando `p.listAvailablePlanes()`.
- Cualquier archivo URDF (Universal Robot Description Format) que cargues puede ser tu "plano" si lo usas como base estática.

¿Cómo explorar qué hay disponible?

1. **Exploración Conceptual (pizarra):** La carpeta a la que apunta `pybullet_data.getDataPath()` es tu "biblioteca por defecto".
2. **Exploración Práctica (código):** Podemos usar Python para listar los archivos de esa carpeta.

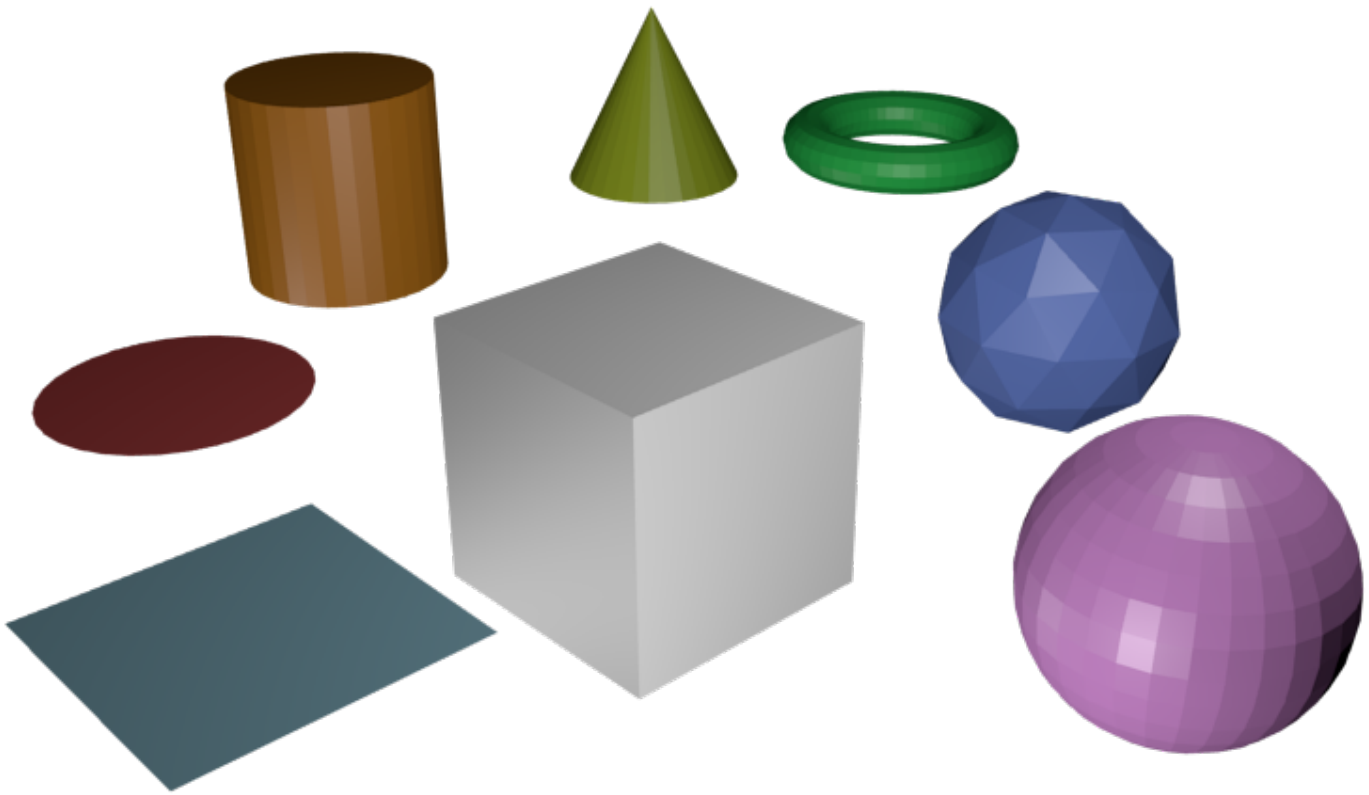
Script para Listar Modelos URDF

```python
import os
import pybullet_data

# Obtener la ruta de datos por defecto
path_data = pybullet_data.getDataPath()

# Listar todos los archivos .urdf en esa carpeta
modelos = [f for f in os.listdir(path_data) if f.endswith(".urdf")]

print("Modelos URDF disponibles en pybullet_data:")
for modelo in modelos:
    print("- " + modelo)
```

Análisis:

- Este script (que no es parte de la simulación) te permite conocer los nombres exactos de los modelos incluidos.

Objetos Primitivos



¿Qué es realmente una Colisión en Simulación?

El Mito del Objeto Sólido

- En la vida real, la materia no puede ocupar el mismo espacio que otra materia.
- En un entorno virtual, **todo es un holograma (un fantasma)** por defecto.
- Si creamos un objeto solo con su `VisualShape` (Forma Visual), este caerá traspasando el suelo infinitamente. El motor gráfico lo dibuja, pero el motor físico lo ignora.

Definición Clave

Una **Colisión** es una restricción matemática. Es una orden que le damos al motor físico que dice: *"Bajo ninguna circunstancia permitas que los vértices de la Geometría A crucen los vértices de la Geometría B"*.

¿Cómo calcula el motor una Colisión?

Calcular si dos mallas 3D complejas se están tocando requiere millones de operaciones matemáticas por segundo. Para optimizar esto, PyBullet utiliza **Volúmenes Envolventes (Bounding Boxes)**:

1. **Fase Amplia (Broadphase):** El motor dibuja "cajas invisibles" alineadas a los ejes alrededor de todos los objetos. Si estas cajas simples no se tocan, el motor descarta la colisión instantáneamente, ahorrando CPU.
2. **Fase Estrecha (Narrowphase):** Solo si las cajas invisibles se intersectan, el motor calcula matemáticamente si las geometrías exactas (esferas, cilindros o cajas) están chocando.

Por esto usamos formas primitivas: Es infinitamente más rápido calcular la intersección matemática de dos esferas (midiendo la distancia entre sus centros) que la de dos modelos 3D hiperrealistas.

Por qué las Colisiones son Vitales en Robótica

Sin un modelo de colisiones riguroso, la simulación robótica no tiene valor de ingeniería. Las necesitamos para:

- **Locomoción:** Un robot móvil solo avanza porque sus ruedas "chocan" contra el suelo y generan tracción. Sin colisión, las ruedas girarían en el aire.
- **Manipulación:** Para que un brazo robótico recoja un objeto, las pinzas (grippers) deben colisionar con las caras del objeto para aplicar fuerza de agarre.
- **Planificación de Trayectorias:** Entrenar a un algoritmo para mover un brazo de un Punto A a un Punto B evitando obstáculos (ej. no chocar contra una mesa de laboratorio).

Creación de Objetos: El Paradigma Visual vs. Físico

¿Por qué no creamos un objeto con un solo comando?

En simulación robótica, procesar la física es computacionalmente costoso. Por ello, PyBullet divide cada objeto en dos "almas":

- **Forma de Colisión (Collision Shape):** Es invisible. Es una geometría matemática simplificada (cajas, esferas) que el motor usa para calcular impactos, gravedad y fricción.
- **Forma Visual (Visual Shape):** Es lo que ven nuestros ojos. Puede tener colores, texturas o mallas 3D complejas. No afecta la física.

Nota Arquitectónica

Para crear una simple caja en el simulador, debemos definir su forma física, su forma visual, y luego "ensamblarlas" en un cuerpo único.

Paso 1: Definiendo las Geometrías Base

```
# 1. Definir la forma Fisica (Colision) de un cubo
colBoxId = p.createCollisionShape(shapeType=p.GEOM_BOX,
                                  halfExtents=[0.5, 0.5, 0.5])

# 2. Definir la forma Visual (Grafica) del mismo cubo
visBoxId = p.createVisualShape(shapeType=p.GEOM_BOX,
                               halfExtents=[0.5, 0.5, 0.5],
                               rgbaColor=[1, 0, 0, 1]) # Color Rojo
```

Análisis de los Parámetros: `shapeType`: Le indica al motor qué figura primitiva usar:

- `p.GEOM_BOX`
- `p.GEOM_SPHERE`
- `p.GEOM_CYLINDER`

Paso 1: Definiendo las Geometrías Base

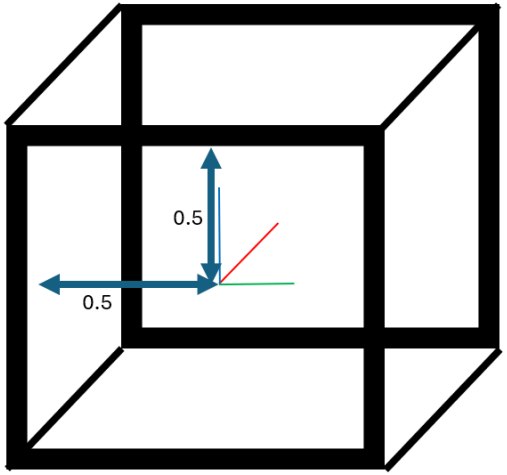


Figure: Cubo de 1 x 1 x 1 metros.

Análisis de los Parámetros:

- `halfExtents`: En motores 3D, el tamaño de las cajas se mide desde el centro geométrico hacia los bordes. Un valor de $[0.5, 0.5, 0.5]$ crea un cubo de $1 \times 1 \times 1$ metros.

Paso 1: Definiendo las Geometrías Base

Color	Nombre	RGB	Hexadecimal
	Negro	(0,0,0)	#000000
	Blanco	(255,255,255)	#FFFFFF
	Rojo	(255,0,0)	#FF0000
	Lima	(0,255,0)	#00FF00
	Azul	(0,0,255)	#0000FF
	Amarillo	(255,255,0)	#FFFF00
	Cian	(0,255,255)	#00FFFF
	Magenta	(255,0,255)	#FF00FF
	Plata	(192,192,192)	#C0C0C0
	Gris	(128,128,128)	#808080

Figure: Ejemplo de Colores

Análisis de los Parámetros:

- `rgbaColor`: Define el color en formato Rojo, Verde, Azul y Transparencia (Alpha), con valores de 0 a 1.

Paso 2: Ensamblando el Cuerpo (MultiBody)

Ver 5_creando_geometrias.py

```
# 3. Unir fisicas y graficos en un objeto real en el simulador
cajaId = p.createMultiBody(baseMass=1.0,
                           baseCollisionShapeIndex=colBoxId,
                           baseVisualShapeIndex=visBoxId,
                           basePosition=[0, 0, 2]) # [X, Y, Z]
```

Análisis de los Parámetros:

- `createMultiBody`: Es el comando que finalmente materializa el objeto en nuestro entorno virtual. Toma las geometrías creadas previamente y les da un lugar en el espacio.
- **La Masa es Clave (`baseMass`):**
  - Si masa $= 0$: El objeto se vuelve **estático** (inmóvil). La gravedad no lo afecta, como si fuera una pared de concreto fija.
  - Si masa > 0: El objeto se vuelve **dinámico**. Está sujeto a las leyes de Newton. Nuestro cubo de 1 kg caerá desde 2 metros de altura.

Otras Formas Primitivas: La Esfera y el Cilindro

```
# Crear una Esfera (Usa parametro 'radius')
colSphere = p.createCollisionShape(p.GEOM_SPHERE, radius=0.5)
visSphere = p.createVisualShape(p.GEOM_SPHERE, radius=0.5, rgbaColor=[0,0,1,1])
p.createMultiBody(baseMass=2.0, baseCollisionShapeIndex=colSphere,
                  baseVisualShapeIndex=visSphere, basePosition=[2, 0, 3])

# Crear un Cilindro (Usa 'radius' y 'height')
colCyl = p.createCollisionShape(p.GEOM_CYLINDER, radius=0.3, height=1.0)
visCyl = p.createVisualShape(p.GEOM_CYLINDER, radius=0.3, rgbaColor=[0,1,0,1])
p.createMultiBody(baseMass=1.5, baseCollisionShapeIndex=colCyl,
                  baseVisualShapeIndex=visCyl, basePosition=[-2, 0, 3])
```

El Poder del Código: Generación Procedural

Ver 6_lluvia_geometrica.py

```
# Crear una "lluvia" de 10 cubos usando un bucle For
for i in range(10):
    p.createMultiBody(baseMass=0.5,
                      baseCollisionShapeIndex=colBox,
                      baseVisualShapeIndex=visBox,
                      basePosition=[0, 0, 2 + i]) # Cada cubo un metro mas arriba
```

Análisis Conceptual:

- A diferencia de programas de diseño 3D manuales (como AutoCAD o SolidWorks), usar Python nos permite generar decenas de cuerpos interactuando dinámicamente con solo 4 líneas de código extra.
- Observen que **reciclamos** `colBox` y `visBox`. PyBullet permite que múltiples cuerpos compartan la misma plantilla geométrica en la memoria RAM, optimizando la simulación.